

RAG Document Q&A with Ollama Embeddings + Groq

Ask questions about your research papers using Ollama embeddings, FAISS, and a Groq Llama model.

 Create Document Embeddings

Enter your question about the research papers:

what is transformers

Answer

The Transformer is a neural-network architecture that processes sequences using stacked layers of self-attention and point-wise fully-connected feed-forward networks. It consists of an encoder stack (six identical layers in the original paper) and a decoder stack, each layer containing a multi-head self-attention sub-layer followed by a feed-forward sub-layer. Residual connections and layer-normalization are applied around each sub-layer, and all sub-layers output vectors of dimension ($d_{\text{model}} = 512$). This design allows the model to capture long-range dependencies efficiently and has been shown to generalize well to tasks such as English constituency parsing.

Response time: 1.41 seconds

▼  Document Similarity Search

Document 1

best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also

experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and

Source: research_papers\Attention.pdf

Document 2

tac, T. Rault, R. Louf, M. Funtowicz, et al., Transformers: State-of-the-art natural language processing, in: Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations, 2020, pp. 38–45. 5 [83] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al., Jax: composable transformations of python + numpy programs (2018). 5 [84] S. Li, J. Fang, Z. Bian, H. Liu, Y. Liu, H. Huang, B. Wang, Y. You, Colossal-ai: A unified deep learning system for large-scale parallel training, arXiv preprint arXiv:2110.14883 (2021). 5 [85] J. He, J. Qiu, A. Zeng, Z. Yang, J. Zhai, J. Tang, Fastmoe: A fast mixture-of-expert training system, arXiv preprint arXiv:2103.13262 (2021). 5 [86] L. Huawei Technologies Co., Huawei mindspore ai development framework, in: Artificial Intelligence Technology, Springer, 2022, pp. 137–162. 5

Source: research_papers\LLM.pdf

Document 3

Figure 1: The Transformer - model architecture. The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Document 4

M. Shoenberger, B. Catanzaro, Reducing activation recomputation in large transformer models, Proceedings of Machine Learning and Systems 5 (2023). 10 [130] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, C. Xiong, Codegen: An open large language model for code with multi- turn program synthesis, arXiv preprint arXiv:2203.13474 (2022). 10, 22, 25, 28 [131] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374 (2021). 10, 25, 29 [132] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al., Competition-level code generation with alphacode, Science 378 (6624) (2022) 1092–1097. 10, 23, 25, 29 [133] N. Shazeer, Fast transformer decoding: One write-head is all you need, arXiv preprint arXiv:1911.02150 (2019). 10